

ReadRight 2.0 — Backend Proxy & Secret-Handling Architecture

Milestone 2 · Mihir Patel & Mir Patel · July 2026

The threat model this addresses

A compiled Flutter app is not a black box. An APK is a zip file; a web build ships its Dart-compiled JavaScript directly to the browser. Anything bundled as a Flutter `asset` is extractable by anyone with the installed app or the deployed URL — no reverse engineering skill required beyond `unzip`.

We confirmed this is not hypothetical: the inherited codebase listed `.env` directly under `flutter: assets:` in `pubspec.yaml`, and `azure_assessor.dart` read the Azure key out of it via `flutter_dotenv` at runtime. Whatever key ended up in that file would have shipped inside every build we produced. That is the exact failure mode the course's guardrail — "the Flutter app must never embed the key" — is written to prevent, and it's why the AI story key never goes near `readright_app/`.

Where the key actually lives

- `backend/.env` (gitignored, exists only on whichever machine runs the proxy) holds `OPENAI_API_KEY`.
- `backend/server.js` reads it once at process startup via `process.env.OPENAI_API_KEY` and holds it in memory for the life of the process. It is never logged, never included in a response body, and never echoed back to the client in any error path.
- The Flutter app holds nothing but the proxy's base URL. `story_service.dart` sends a plain JSON body (`words`, `theme`, `gradeLevel`) to `POST /api/story` and gets a story back — no credential material crosses the network to the client, ever, in either direction.

What the proxy actually does

It's intentionally thin: one route, input validation, one outbound call.

```
Flutter app --JSON (words, theme)--> backend/server.js --Bearer key--> OpenAI
          <--JSON (story only)--          <--full response--
```

- Validates the word list (non-empty, capped at 12 words) before making any upstream call — this is also our cost guardrail, since it bounds worst-case token usage per request.
- Builds the Dolch-based prompt server-side, so prompt construction (and the mini-tier model choice) is centrally controlled and can't be overridden by the client.
- Only forwards the extracted `story` string back to the app — never the raw OpenAI response, which would otherwise leak model/version details unnecessarily.

How it stays safe across our GitHub workflow

- `.gitignore` has a repo-wide `*.env` pattern, so `backend/.env`, `readright_app/.env`, and the root `.env` are all excluded regardless of which directory someone runs `git add` from — a stray `git add -A` can't stage a secret by accident.
- The key was distributed out-of-band, through the private team Canvas announcement, and each of us populates our own local `backend/.env`. It has never been pasted into a commit message, source file, or comment — we grepped the git history before writing this document to confirm.
- We deliberately did not commit a real key anywhere, including in scratch/test commits, which is the easiest way secrets accidentally end up permanently in history even after being "removed" in a later commit.

Why this matters more than it might seem: if `backend/.env` leaks (e.g., a compromised laptop), only that one key is exposed, and we rotate it on OpenAI's dashboard with zero app updates required. If the old embed-in-the-binary pattern had shipped with a real key, every installed copy of the app would carry a permanently extractable credential — rotating the key wouldn't help until every user updated, and by then the leaked key had already been usable indefinitely. The proxy converts an unbounded, unpatchable exposure into a bounded, rotatable one.

Known limitation, and the plan for it

The M2 proxy has no auth of its own — anything that can reach `localhost:8787` can call `/api/story`. That's acceptable for a local de-risk spike proving the pipeline works, but not for a deployed M3 proxy. Before deployment we will verify the caller's Firebase ID token on the proxy (Firebase Auth is already used app-wide, so this is additive, not new infrastructure) so only an authenticated teacher session can trigger generation, and we'll lock CORS down to our deployed app's origin instead of the open default we used for local development.